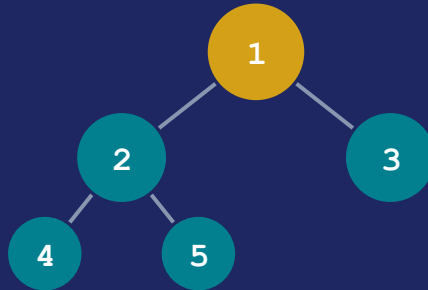


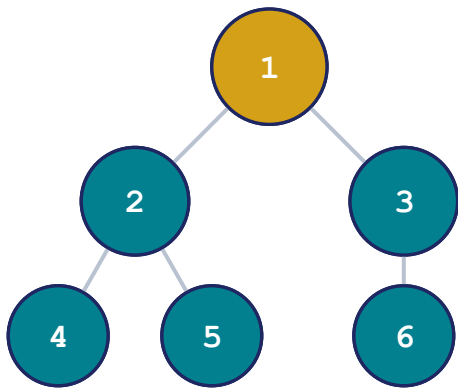
Storing a Tree in Memory

Complete Binary Trees, Array and Linked Representations

Unit III • Lecture 22



Complete binary tree: filled level by level



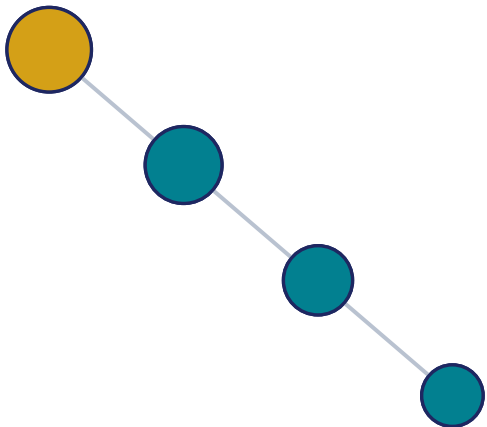
6 nodes: last level filling LEFT to RIGHT, no gaps

The precise rule

Every level is COMPLETELY filled, except possibly the last — and the last level fills strictly LEFT to RIGHT with no gaps.

A gap anywhere before the rightmost filled position breaks completeness.

Why the shape matters: array space, verified



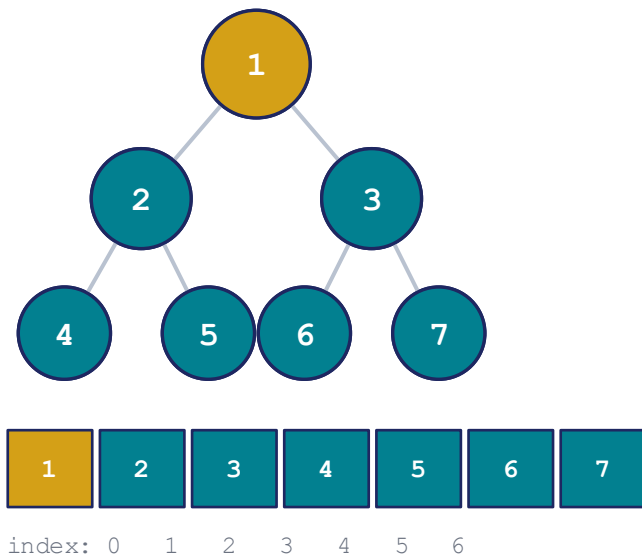
4-node right-skewed tree

Verified by index arithmetic

Right child of index i is $2i+2$. Following right, right, right from the root: indices 0, 2, 6, 14.

An array must be sized 15 to hold index 14 — for only 4 real nodes.
11 slots wasted.

Array representation: three formulas, no pointers



For node at index i

left child = $2i + 1$

right child = $2i + 2$

parent = $(i-1) / 2$

(integer division for parent)

No next pointer anywhere — arithmetic replaces it.

Verified: every parent/child relationship, by formula

index	value	parent	left	right
0	1	none (root)	1 (2)	2 (3)
1	2	0 (1)	3 (4)	4 (5)
2	3	0 (1)	5 (6)	6 (7)
3	4	1 (2)	none	none

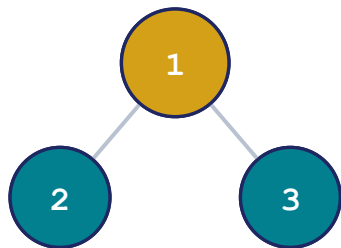
(values in parentheses show what is STORED at that index — the formula gives the index, tree[index] gives the value)

This is the program's real output, compiled and run — every row computed by parentIndex/leftIndex/rightIndex, never hand-filled.

Linked representation: two pointers per node

```
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

- Same struct shape as every earlier Node — but TWO pointer fields instead of one next.
- left and right point at child SUBTREES, or NULL if that side is empty.
- No index arithmetic at all — the shape is whatever the pointers say it is, complete or not.



```
root->left == node(2)
root->right == node(3)
```

Building the same 7-node tree, by hand

```
struct Node *newNode(int value)
{
    struct Node *n = malloc(sizeof(*n));
    if (n == NULL) exit(1);
    n->data = value;
    n->left = NULL;
    n->right = NULL;
    return n;
}
```

```
struct Node *root = newNode(1);
root->left = newNode(2);
root->right = newNode(3);
root->left->left = newNode(4);
root->left->right = newNode(5);
root->right->left = newNode(6);
root->right->right = newNode(7);
```

- newNode mirrors every earlier Node constructor — malloc, fill, NULL out the pointers.
- "root->left->left" reads exactly like walking down the tree: root, then its left child, then THAT node's left child.
- No formula needed — the chain of arrows in the code IS the chain of arrows in the diagram.

Array vs Linked — the trade-off, one more time

Aspect	Array Representation	Linked Representation
Navigation	Index formulas ($2i+1$, $2i+2$)	Pointers (left, right)
Best suited to	COMPLETE trees	Any shape, including skewed
Wasted memory	Can be large for skewed trees	None — exactly as many nodes as exist
Insert a new node	Next free index, $O(1)$ if known	malloc + one pointer assignment
Extra memory per node	None	Two pointers (left, right)

Two ways to store the same shape

- Complete binary tree: every level full except possibly the last, which fills strictly left to right.
- Array representation: for index i , $\text{left} = 2i+1$, $\text{right} = 2i+2$, $\text{parent} = (i-1)/2$ — no pointers, but efficient only for complete trees.
- A skewed tree wastes massive array space (verified: 4 nodes needing a 15-slot array) — shape drives the cost.
- Linked representation: `struct Node { data; left; right; }` — exactly as much memory as nodes exist, any shape, at the cost of two pointers per node.

EXIT TICKET A complete binary tree is stored as `tree[0..10]` (11 nodes). Using the formulas, find the parent, left child, and right child of index 4.